FileRelic — Current Implementation (reference-diagram layout)

The GUI currently invokes the CLI and presents its output as readable cards.
It does not consume InspectionReport directly yet.

User interface

«Qt class» MainWindow

- -selectedImagePath: QString
- -rawReport: QString
- +chooseImage()
- +analyzeImage()
- +appendOutput()
- +inspectionFinished(exitCode, exitStatus)
- +toggleRawLog(visible)
- -cliPath(): QString
- -inspectImage(path)
- -renderResults(succeeded)
- -clearResults()
- -addResultCard(...)

starts process

«function» CLI entry point

- +dispatch help / version / inspect / demo
- +main(argc, argv)

«Qt class» QProcess

builds result cards

External framework

«framework» Qt Widgets

launches

runs

«process» filerelic executable

inspect command

demo fixtures

This shared report currently stores image status and rendered text.
Typed partition and filesystem result collections are future work.

Report and output

«free function» Report rendering API

- +render_report(report, output)

formats

«struct» InspectionReport

- +image_path: filesystem::path
- +image_size_bytes: uintmax_t
- +exit_code: int
- +rendered_text: string
- +succeeded(): bool

returns

«free function» Inspection API

- +inspect_image(path): InspectionReport

performs inspection

Image inspection

«free functions» Partition table functions

- inspect_image_file(path)
- inspect_mbr(image, size)
- inspect_extended_partition(...)
- inspect_gpt(image, size)
- read_gpt_header(...)
- read_partition_entry(...)

validates headers

«struct» GptHeader

- +header_size: uint32
- +current_lba: uint64
- +backup_lba: uint64
- +first_usable_lba: uint64
- +last_usable_lba: uint64
- +partition_entries_lba: uint64
- +partition_entry_count: uint32
- +partition_entry_size: uint32
- +partition_entries_crc32: uint32

reads entries

«struct» MbrEntry

- +type: unsigned char
- +start_sector: uint32
- +sector_count: uint32

reads and validates bytes

«free functions» Binary and image helpers

- read_sector(...)
- read_bytes_at(...)
- matches_bytes_at(...)
- read_little_endian_u32(...)
- read_little_endian_u64(...)
- crc32_ieee(...)
- read_ascii_field(...)
- format_uuid(...)

decodes metadata

detects filesystem

«free functions» Filesystem functions

- detect_file_system(...)
- print_file_system_report(...)
- inspect_ext_superblock(...)
- inspect_ext_root_directory(...)
- inspect_ext_file_inode(...)

fills metadata

«struct» ExtSuperblockInfo

- +block_size: uint32
- +inode_size: uint16
- +inodes_per_group: uint32

Solid boxes are C++ classes or structs that exist in the code.
Boxes marked «free functions» group real functions; they are not classes.
This diagram follows the supplied class.jpeg's left-to-right flow while
describing the current implementation, including its present CLI-based GUI link.